



Wydział Informatyki

Informatyka, 8 semestr

Środowiska Udostępniania Usług

Application Observability

Kamil Rudny

Filip Jędrzejewski

Mateusz Grzybek

Marta Bukowińska

Kraków, 2026

Spis treści

1	Podstawy teoretyczne	4
1.1	Natywne aplikacje chmurowe	4
1.2	Obserwowalność aplikacji	4
1.3	Wizualizacja danych	5
1.4	Wykorzystane technologie	6
1.4.1	Grafana	7
2	Koncepcja	8
2.1	Architektura systemu	8
2.2	Aplikacja	8
2.3	Obserwowalność aplikacji	9
2.4	Wizualizacja	9
3	Realizacja	10
3.1	Architektura	10
3.1.1	Warstwa infrastruktury	10
3.1.2	Warstwa orkiestracji	10
3.1.3	Aplikacja	10
3.1.4	Warstwa obserwowalności	11
3.1.5	Warstwa przechowywania danych	11
3.1.6	Warstwa wizualizacji	11
3.1.7	Warstwa integracji z agentem AI	12
3.2	Środowisko	12
3.2.1	Środowisko uruchomieniowe	12
3.2.2	Konfiguracja klastra Kubernetes	12
3.2.3	Dostęp do usług	13
3.2.4	Zasoby i parametry środowiska	13
3.2.5	Wykorzystane narzędzia	13
3.2.6	Generowanie ruchu testowego	13
3.3	Instalacja systemu	14
3.3.1	Inicjalizacja infrastruktury	14
3.3.2	Wdrożenie komponentów Kubernetes	14
3.3.3	Instalacja MCP Server	14
4	Rezultaty	15
4.1	Infrastruktura	15
4.2	Wdrożenie komponentów	16
4.2.1	Stos obserwowalności	16

4.2.2	Aplikacja Online Boutique	18
4.3	Integracja z agentem AI	19
4.4	Dashboard Grafana	22
4.5	Generowanie ruchu testowego	23
4.6	Proces instalacji	25
4.7	Struktura repozytorium	25
5	Podsumowanie	26
5.1	Możliwości rozwoju	26

Wprowadzenie

Współczesne systemy informatyczne opierają się na architekturze rozproszonej z wykorzystaniem niezależnych mikroserwisów wdrażanych jako skonteneryzowane aplikacje na klastrach. W takim środowisku awaria pojedynczego komponentu może kaskadowo ograniczać działanie pozostałych, wpływając negatywnie na funkcjonalność oraz dostępność całej platformy. Brak odpowiednich narzędzi do obserwacji zmian w stanie serwisów sprawia, że zlokalizowanie źródła problemu staje się czasochłonne i kosztowne.

Projekt dotyczy wdrożenia obserwowalności aplikacji w środowisku opartym na orkiestratorze konteneryzacji Kubernetes. Jako kolektor telemetry wykorzystano Grafana Alloy, czyli dystrybucję OpenTelemetry Collector rozwijaną przez Grafana Labs. Alloy agreguje metryki, logi, ślady i profile w jednym narzędziu, co pozwala zastąpić kilka niezależnych kolektorów jedną, spójną konfiguracją.

Celem projektu jest wdrożenie kompletnego środowiska uruchomieniowego dla aplikacji z uwzględnieniem aspektu obserwowalności poprzez zbieranie i przetwarzanie danych telemetrycznych oraz ich wizualizację w dashboardach Grafany. Komponenty współtworzące projekt są wdrażane do klastra Kubernetes, co pozwala na łatwe skalowanie horyzontalne i odtwarzanie środowiska.

Rozdział 1

Podstawy teoretyczne

Rozdział porusza kwestie teoretyczne projektu. Wyjaśnia pojęcia natywnych aplikacji chmurowych, obserwowalności, wizualizacji oraz porusza kwestie głównych wykorzystanych narzędzi.

1.1 Natywne aplikacje chmurowe

Natywne aplikacje chmurowe (ang. *cloud native applications*) [10] to aplikacje, które zostały zaprojektowane, zbudowane i zarządzane tak, aby w pełni wykorzystać potencjał uruchomienia w chmurze.

Opierają się one na czterech głównych filarach:

1. **Mikroserwisy** – zamiast jednej wielkiej aplikacji (tak zwanego monolitu), system składa się z wielu mniejszych, niezależnych usług. Każda odpowiada za konkretną funkcjonalność i komunikuje się z innymi przez interfejs API. Dzięki temu awaria jednej części nie skutkuje awarią całego systemu.
2. **Konteneryzacja** – każdy mikroserwis jest umieszczany w kontenerze (na przykład Docker) wraz ze wszystkim, czego potrzebuje do działania (czyli przykładowo kod, biblioteki). Gwarantuje to, że aplikacja będzie działać identycznie na komputerze programisty, serwerze testowym i w chmurze produkcyjnej.
3. **Orkiestracja** – Zarządzanie setkami kontenerów ręcznie jest niemożliwe lub bardzo trudne i narażone na wysokie ryzyko błędu. Narzędzia do orkiestracji (takie jak na przykład Kubernetes) w sposób automatyczny dbają o utrzymanie zadeklarowanych kontenerów w pożądanym stanie poprzez ich uruchamianie, restartowanie w przypadku awarii i skalowanie.
4. **Automatyzacja** – odpowiada za minimalizowanie błędu ludzkiego podczas zmian w systemie i zarządzaniu nim.

1.2 Obserwowalność aplikacji

Obserwowalność (ang. *observability*) [9] to zdolność do wnioskowania o wewnętrznym stanie systemu na podstawie jego zewnętrznych sygnałów. System jest obserwowalny, gdy na

podstawie emitowanych danych można odpowiedzieć na dowolne pytanie o jego działaniu, nawet takie, którego nie przewidziano z góry.

Obserwowalność opiera się na trzech filarach:

- **Metryki** (ang. *metrics*) – wartości liczbowe agregowane w czasie, opisujące stan systemu (np. zużycie CPU, liczba żądań HTTP na sekundę, opóźnienie odpowiedzi). Metryki pozwalają szybko wykryć anomalie i są podstawą alertów.
- **Logi** (ang. *logs*) – chronologiczne zapisy zdarzeń generowanych przez aplikację. Zawierają kontekst, który ułatwia zrozumienie przyczyny konkretnego błędu lub nietypowego zachowania.
- **Ślady** (ang. *traces*) – reprezentacja przepływu pojedynczego żądania przez kolejne serwisy w systemie rozproszonym. Każdy ślad składa się ze spanów, które odpowiadają operacjom wykonywanym w poszczególnych komponentach. Dzięki śladom można precyzyjnie wskazać, który serwis wprowadza opóźnienie lub powoduje błąd.

Poza tymi trzema filarami coraz częściej wyróżnia się także **profile**, które dostarczają informacje o zużyciu zasobów na poziomie kodu źródłowego, np. które funkcje konsumują najwięcej czasu procesora lub pamięci.

Kluczową rolę w ekosystemie obserwowalności odgrywa standard **OpenTelemetry** [7]. Jest to otwarty framework nadzorowany przez Cloud Native Computing Foundation, który definiuje jednolite API, SDK oraz protokół do instrumentacji aplikacji i eksportu danych telemetrycznych. OpenTelemetry jest niezależny od dostawcy backendu, co oznacza, że zinstrumentowana aplikacja może wysyłać dane zarówno do Grafany, jak i do Jaegera, Datadoga czy dowolnego innego zgodnego systemu.

1.3 Wizualizacja danych

Surowe dane telemetryczne mają ograniczoną wartość, jeśli nie można ich szybko przeanalizować. Wizualizacja przekształca strumień metryk, logów i śladów w czytelne wykresy, tabele i mapy, dzięki czemu zespoły operacyjne mogą podejmować decyzje w ciągu zaledwie sekund zamiast minut.

Grafana [3] jest open-source'ową platformą do wizualizacji i monitoringu, która łączy się z wieloma źródłami danych. Dane prezentowane są na interaktywnych dashboardach, które składają się z paneli. Każdy panel to niezależna wizualizacja, np. wykres szeregów czasowych, histogram, tabela logów albo mapa śladów.

Ważnym elementem jest możliwość korelacji sygnałów. Grafana pozwala przechodzić bezpośrednio z wykresu metryki do odpowiadających jej logów, a z logu do konkretnego śladu rozproszonego. Taka nawigacja drastycznie skraca czas diagnozy incydentów.

Alerting w Grafanie umożliwia definiowanie reguł opartych na zapytaniach do źródeł danych. Gdy warunek alertu zostanie spełniony, powiadomienie trafia na wybrany kanał: e-mail, Slack, PagerDuty lub inny skonfigurowany kontakt.

1.4 Wykorzystane technologie

Kubernetes

Kubernetes (K8s) [6] to silnik orkiestracji kontenerów, rozwijany pierwotnie przez Google, a obecnie zarządzany przez Cloud Native Computing Foundation. Automatyzuje wdrażanie, skalowanie i zarządzanie skonteneryzowanymi aplikacjami.

Podstawowe zasoby orkiestratora Kubernetes istotne w kontekście projektu:

- **Pod** – najmniejsza jednostka wdrożeniowa, reprezentująca pojedynczy kontener lub grupę ściśle powiązanych ze sobą kontenerów uruchomionych na jednym węźle,
- **Deployment** – zasób służący do wdrażania podów z zadeklarowanym stanem (liczba replik, obraz kontenera, zmienne środowiskowe). Kubernetes automatycznie dąży do utrzymania działających zasobów w stanie zgodnym ze zdefiniowanym,
- **Service** – zasób wykorzystywany do przekierowywania ruchu do wybranych podów. Każdy serwis posiada własną nazwę DNS, za pomocą której jest odkrywalny w całym klastrze. Serwis udostępnia podstawową funkcjonalność rozkładu obciążenia (load balancing),
- **DaemonSet** – zasób wykorzystywany do uruchamiania dokładnie jednego poda na każdym węźle klastra. Jest to typowy sposób wdrażania kolektorów telemetry, w tym Grafana Alloy,
- **ConfigMap i Secret** – zasoby wykorzystywane do przechowywania odpowiednio konfiguracji i danych wrażliwych w postaci par klucz — wartość, które mogą następnie zostać wczytane jako zmienne środowiskowe dla kontenerów,
- **Helm** – menedżer pakietów dla Kubernetesa, umożliwiający wdrażanie złożonych aplikacji za pomocą sparametryzowanych szablonów (chartów).

Grafana Alloy

Grafana Alloy [4] to open-source’owy kolektor telemetry, będący dystrybucją OpenTelemetry Collector z wbudowanymi pipeline’ami Prometheusa. Zastępuje wcześniejszy projekt Grafana Agent.

Alloy zbiera metryki, logi, ślady i profile w ramach jednego procesu, co eliminuje potrzebę uruchamiania osobnych kolektorów dla każdego typu sygnału. Konfiguracja opiera się na deklaratywnym języku (Alloy configuration language), w którym podstawową jednostką jest **komponent**. Każdy komponent realizuje jedno zadanie, np. odbieranie danych OTLP, scrapowanie metryk Prometheusa albo eksport logów do Loki.

Alloy wspiera klasteryzację, co pozwala rozłożyć obciążenie na wiele instancji. W środowisku Kubernetes typowo wdraża się go jako DaemonSet (jedna pod na węzeł) lub jako Deployment (centralna instancja przetwarzająca).

Prometheus

Prometheus [8] to system monitoringu i baza danych szeregów czasowych. Działa w modelu pull: okresowo odpytuje (scrapuje) endpointy aplikacji i zapisuje zebrane metryki.

Definiuje własny format ekspozycji metryk oraz język zapytań PromQL, który jest standardem w ekosystemie cloud-native.

1.4.1 Grafana

Grafana [3] jest otwartą platformą służącą do interaktywnej wizualizacji, monitorowania oraz analizy danych pochodzących z wielu różnorodnych źródeł. System ten umożliwia integrowanie metryk, logów oraz śladów (traces) w ujednoliconych, dynamicznych pulpitych (dashboards).

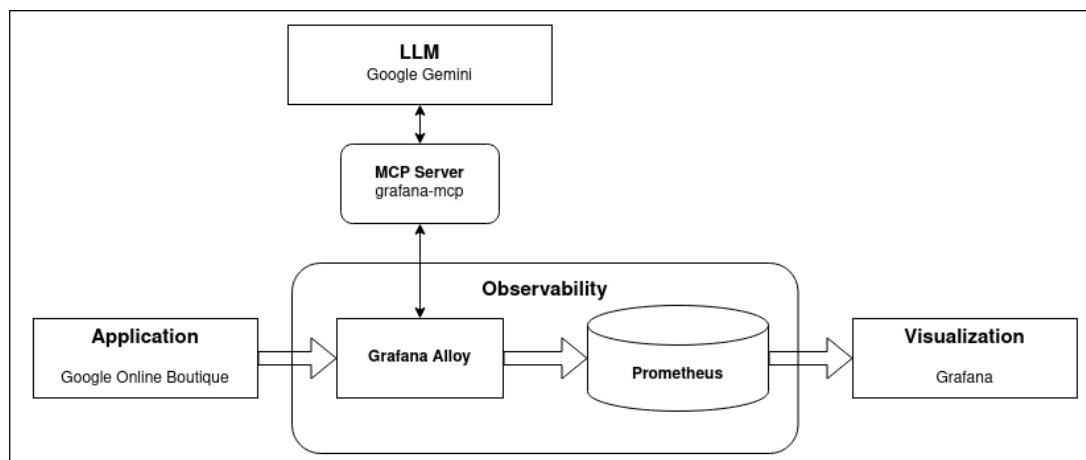
Rozdział 2

Koncepcja

W tym rozdziale zostanie opisana koncepcja systemu, który jest celem tego projektu. Zostaną omówione kwestie wysokopoziomowej architektury, wykorzystanej aplikacji, komponentu observability, wykorzystania sztucznej inteligencji oraz wizualizacji.

2.1 Architektura systemu

Grafika poniżej przedstawia wysokopoziomową architekturę systemu. Poszczególne komponenty zostały bardziej szczegółowo opisane w paragrafach poniżej.



Rysunek 2.1: Wysokopoziomowa architektura systemu

2.2 Aplikacja

W ramach realizacji projektu wykorzystana została gotowa aplikacja oparta na architekturze mikrosług Online Boutique [2], opracowana i udostępniona przez firmę Google jako demonstracja aplikacji mikroserwisowej, przeznaczonej do uruchamiania na dowolnym klastrze Kubernetes. Aplikacja symuluje działanie sklepu internetowego, w którym użytkownik może przeglądać produkty, dodawać je do koszyka oraz składać zamówienia.

Aplikacja składa się z 11 mikrosług, każda realizująca odrębną funkcję biznesową. Usługi zostały napisane w różnych językach programowania, a do komunikacji między-serwisowej

wykorzystują protokół gRPC stworzony przez Google. Taki stos technologiczny pozwala w odpowiedni sposób zasymulować rzeczywiste, chmurowe środowisko produkcyjne wykorzystujące jednocześnie szeroki zakres technologii.

Wśród mikrousług działających w ramach aplikacji można wymienić między innymi usługę frontendową udostępniającą interfejs graficzny użytkownika, usługę odpowiedzialną za logikę koszyka zakupowego z wykorzystaniem bazy danych in-memory Redis oraz usługę zawierającą katalog produktów.

2.3 Obserwowalność aplikacji

Obserwowalność aplikacji będzie realizowana z wykorzystaniem Grafana Alloy, kontrolowanego za pomocą agenta AI (**Gemini**) [1] przez oficjalny MCP Server dla Grafany, czyli `mcp-grafana` [5]. Taki wybór narzędzi umożliwi nam względnie prostą integrację systemu.

Użycie agenta AI pozwoli nam na kontrolę stanu aplikacji za pomocą języka naturalnego. Oznacza to, na przykład, możliwość:

- inteligentnej analizy zbieranych danych
- aktualizacji konfiguracji Grafana Alloy
- zapytania o ogólny stan aplikacji w formie zdania: «Sprawdź aktualny stan aplikacji.»
- w przypadku zauważenia anomalii w zachowaniu aplikacji, zapytania «Mamy problem XYZ, sprawdź co się dzieje i zdiagnozuj przyczynę.»

2.4 Wizualizacja

Komponent wizualizacji, oparty na Grafanie połączonej z Prometheusem, będzie miał za zadanie umożliwić nam podgląd na aktualny stan aplikacji. Pozwoli nam to na kontrolę zaimplementowanych rozwiązań: zarówno stanu aplikacji, jak i efektywności działań agenta AI.

Rozdział 3

Realizacja

Ten rozdział jest poświęcony szczegółowym aspektom implementacji i wdrożenia naszej koncepcji. Zostanie omówiona szczegółowa architektura systemu, następnie przedstawione zostanie środowisko użyte do uruchomienia, po czym opisane zostaną kwestie instalacji i konfiguracji systemu. Rozdział zamknie paragraf opisujący przygotowanie danych, na których operować będzie aplikacja.

3.1 Architektura

Architektura systemu została podzielona na kilka logicznych warstw odpowiadających za infrastrukturę, orkiestrację, aplikację, obserwowalność, wizualizację oraz integrację z agentem AI.

3.1.1 Warstwa infrastruktury

System uruchamiany jest w środowisku chmurowym AWS na pojedynczej instancji EC2. Na maszynie tej działa lekka dystrybucja Kubernetes **K3s**, która zapewnia orkiestrację wszystkich komponentów.

3.1.2 Warstwa orkiestracji

K3s działające na EC2 stanowi bazę dla wszystkich komponentów systemu. System podzielony jest na dwie przestrzenie nazw:

- **observability** — zawiera komponenty observability stack: Prometheus, Loki, Tempo, Grafana, Grafana Alloy oraz MCP Server
- **boutique** — zawiera Online Boutique i jej 11 mikroserwisów

3.1.3 Aplikacja

W systemie wykorzystano aplikację Online Boutique jako warstwę usługową wdrożoną w klastrze Kubernetes w postaci zestawu niezależnych mikroserwisów. Aplikacja pełni rolę środowiska testowego oraz źródła danych telemetrycznych dla warstwy obserwowalności

Integracja aplikacji z pozostałymi komponentami systemu obejmuje:

- realizację komunikacji między mikroserwisami w ramach przetwarzania żądań pochodzących od użytkowników oraz skryptu testowego,
- emisję logów i śladów do warstwy obserwowalności,
- ekspozycję interfejsu użytkownika poprzez usługę typu NodePort,
- integrację z OpenTelemetry poprzez konfigurację zmiennych środowiskowych w wybranych mikroserwisach.

3.1.4 Warstwa obserwowalności

Warstwa obserwowalności realizuje proces zbierania i przetwarzania danych telemetrycznych z aplikacji oraz klastra Kubernetes.

Centralnym elementem tej warstwy jest Grafana Alloy, działający jako agent kolekcji danych telemetrycznych. Alloy został wdrożony w klastrze Kubernetes i odpowiada za integrację pomiędzy źródłami danych a systemami ich przechowywania.

W ramach działania Alloy realizowane są następujące zadania:

- automatyczne wykrywanie podów i usług w klastrze Kubernetes,
- zbieranie logów aplikacyjnych oraz systemowych i przekazywanie ich do Loki,
- odbieranie śladów rozproszonych, generowanych przez mikroserwisy aplikacji Online Boutique w standardzie OpenTelemetry (OTLP),
- zbieranie metryk z aplikacji oraz komponentów Kubernetes,
- wstępne przetwarzanie i agregacja danych telemetrycznych przed ich dalszym przekazaniem.

3.1.5 Warstwa przechowywania danych

Dane telemetryczne są składowane w trzech niezależnych systemach:

- Prometheus — dla metryk,
- Loki — dla logów,
- Tempo — dla śladów.

3.1.6 Warstwa wizualizacji

Warstwa wizualizacji została zrealizowana z wykorzystaniem Grafany, która pełni rolę interfejsu do zapytań i prezentacji danych telemetrycznych. Grafana jest połączona z trzema źródłami danych: Prometheus (metryki), Loki (logi) oraz Tempo (traces), z którymi komunikuje się poprzez skonfigurowane datasources.

W systemie Grafana wykonuje zapytania do dedykowanych backendów i prezentuje ich wyniki w formie dashboardów.

Integracja danych odbywa się na poziomie modelu telemetrycznego oraz konfiguracji źródeł danych. W szczególności wykorzystywane są:

- identyfikatory trace (`trace_id`) propagowane w logach i trace'ach,
- exemplars w Prometheus umożliwiające powiązanie metryk z trace'ami,
- mechanizmy cross-linking w Grafanie umożliwiające przechodzenie pomiędzy widokami metryk, logów i trace'ów.

Dzięki temu Grafana umożliwia spójną eksplorację danych telemetrycznych w ramach jednego interfejsu, mimo że dane są przechowywane w niezależnych systemach backendowych.

3.1.7 Warstwa integracji z agentem AI

Integracja systemu z agentem AI została zrealizowana przy użyciu Grafana MCP Server, który udostępnia funkcjonalności Grafany poprzez protokół Model Context Protocol (MCP).

MCP Server pełni rolę warstwy pośredniej pomiędzy agentem AI a Grafaną, umożliwiając komunikację poprzez API Grafany.

Agent AI może:

- wykonywać zapytania do danych telemetrycznych (metryki, logi, traces),
- przeglądać i analizować dashboardy Grafany,
- wykorzystywać dane do diagnozy stanu systemu.

3.2 Środowisko

3.2.1 Środowisko uruchomieniowe

System został wdrożony w środowisku chmurowym Amazon Web Services (AWS). Głównym elementem infrastruktury jest instancja EC2 z systemem Amazon Linux 2023, na której uruchomiono klaster Kubernetes k3s.

Infrastruktura tworzona jest automatycznie przy użyciu Terraform, co pozwala na powtarzalne przygotowanie środowiska testowego oraz łatwą rekonfigurację parametrów wdrożenia, takich jak typ instancji czy region.

W projekcie wykorzystano instancję typu `t3.large` z wolumenem EBS `gp3` o pojemności 30 GB. Konfiguracja sieciowa umożliwia dostęp do usług klastra, interfejsu Grafany oraz aplikacji demonstracyjnej.

3.2.2 Konfiguracja klastra Kubernetes

Na instancji EC2 uruchamiany jest klaster k3s pełniący rolę warstwy orkiestracji systemu.

Konfiguracja klastra obejmuje:

- generowanie certyfikatów TLS dla publicznego i prywatnego adresu IP instancji,
- wykorzystanie `local-path provisioner` jako domyślnej klasy storage,

- aktywację metryk komponentów Kubernetes (`kube-controller-manager`, `kube-scheduler`, `kube-proxy`),
- podział środowiska na przestrzenie nazw `observability` oraz `boutique`.

3.2.3 Dostęp do usług

Wybrane komponenty systemu zostały udostępnione poprzez usługi typu NodePort:

- **30080** — frontend aplikacji Online Boutique,
- **30300** — interfejs Grafany,
- **30090** — Grafana MCP Server,
- **6443** — Kubernetes API.

Dostęp administracyjny do instancji realizowany jest za pomocą protokołu SSH.

3.2.4 Zasoby i parametry środowiska

W celu dostosowania systemu do ograniczonych zasobów pojedynczej instancji EC2 zastosowano limity zasobów oraz uproszczoną konfigurację komponentów `observability`.

Najważniejsze parametry środowiska obejmują:

- retencję metryk w Prometheus ustawioną na 3 dni,
- interwał zbierania metryk wynoszący 30 sekund,
- ograniczenia zasobów CPU i pamięci dla Grafana Alloy oraz MCP Server,
- filtrowanie starszych logów w celu ograniczenia obciążenia systemu.

Dodatkowo wyłączono wybrane komponenty aplikacji demonstracyjnej generujące wysokie zużycie zasobów.

3.2.5 Wykorzystane narzędzia

Proces wdrażania i zarządzania środowiskiem opiera się na następujących narzędziach:

- **Terraform** — zarządzanie infrastrukturą jako kodem,
- **Helm** — instalacja komponentów w Kubernetes,
- **kubectl** — administracja klastrem Kubernetes,
- **Bash** — automatyzacja procesu wdrażania.

3.2.6 Generowanie ruchu testowego

W celu generowania danych telemetrycznych wykorzystano skrypt `load.sh`, który wykonuje cykliczne żądania HTTP do wybranych endpointów aplikacji Online Boutique.

Skrypt umożliwia konfigurację liczby równoległych żądań oraz opóźnień pomiędzy kolejnymi zapytaniami, co pozwala na symulowanie ruchu użytkowników i obserwację zachowania systemu pod obciążeniem.

3.3 Instalacja systemu

3.3.1 Inicjalizacja infrastruktury

Proces instalacji rozpoczyna się od uruchomienia skryptu `init.sh`, odpowiedzialnego za przygotowanie środowiska AWS oraz inicjalizację klastra Kubernetes.

Skrypt realizuje następujące kroki:

- instalację wymaganych narzędzi (Terraform, Helm),
- inicjalizację konfiguracji Terraform,
- utworzenie infrastruktury AWS,
- pobranie konfiguracji `kubeconfig`,
- konfigurację lokalnego dostępu do klastra.

3.3.2 Wdrożenie komponentów Kubernetes

Po przygotowaniu środowiska wykonywany jest skrypt `deploy-k8s.sh`, odpowiedzialny za wdrożenie komponentów systemu w klastrze Kubernetes.

Proces obejmuje:

- dodanie repozytoriów Helm,
- utworzenie namespace `observability` i `boutique`,
- instalację Prometheus, Loki, Tempo, Grafana oraz Grafana Alloy,
- wdrożenie aplikacji Online Boutique,
- załadowanie dashboardów Grafany.

3.3.3 Instalacja MCP Server

Integracja z agentem AI realizowana jest przez skrypt `deploy-mcp.sh`, który odpowiada za konfigurację Grafana MCP Server.

Skrypt:

- tworzy konto serwisowe w Grafanie,
- generuje token API,
- zapisuje dane uwierzytelniające w Kubernetes Secret,
- wdraża MCP Server w namespace `observability`.

Rozdział 4

Rezultaty

Rozdział opisuje proces implementacji i wdrożenia systemu opisanego w rozdziale 2. Przedstawiono konfigurację infrastruktury, wdrożenie komponentów, integrację z agentem AI oraz generowanie ruchu testowego.

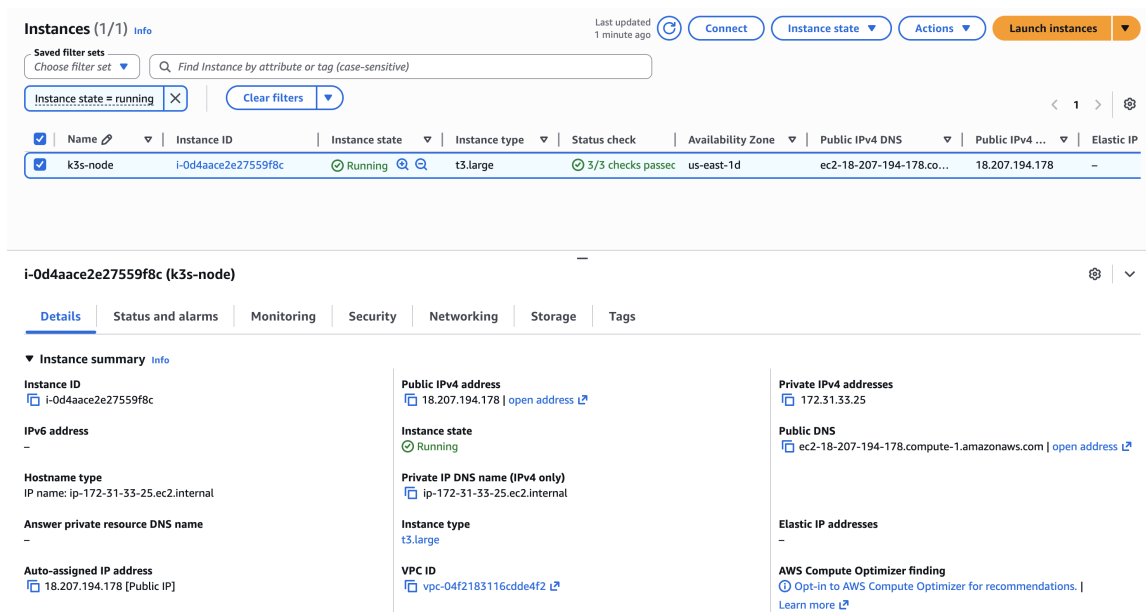
4.1 Infrastruktura

Infrastruktura została zdefiniowana jako kod przy użyciu narzędzia HashiCorp Terraform.

```
1 resource "aws_instance" "k3s" {
2   ami                = data.aws_ami.al2023.id
3   instance_type      = var.instance_type
4   key_name           = aws_key_pair.k3s.key_name
5   vpc_security_group_ids = [aws_security_group.k3s.id]
6   iam_instance_profile = data.aws_iam_instance_profile.lab.name
7
8   root_block_device {
9     volume_size = 30
10    volume_type = "gp3"
11  }
12
13  user_data = file("${path.module}/user_data.sh")
14  tags = { Name = "k3s-node", Project = "SUU-App-0" }
15 }
```

Listing 4.1: Definicja instancji EC2 (main.tf)

Po utworzeniu instancji skrypt `user_data.sh` automatycznie instaluje K3s i konfiguruje klaster tak, aby udostępniał metryki komponentów sterujących dla Prometheusa.



Rysunek 4.1: Instancja EC2 w konsoli AWS

4.2 Wdrożenie komponentów

Wszystkie komponenty systemu wdrażane są automatycznie przez skrypt `deploy-k8s.sh`, który wykorzystuje Helm do instalacji w dwóch przestrzeniach nazw: `observability` i `boutique`.

4.2.1 Stos obserwowalności

Prometheus, Loki, Tempo, Grafana Alloy oraz Grafana zostały zainstalowane z dedykowanymi plikami konfiguracyjnymi (Helm values).

```
1 NODE_IP=$(kubectl get nodes -o \
2   jsonpath='{.items[0].status.addresses
3     [?(@.type=="InternalIP")].address}')
4
5 helm upgrade --install kps \
6   prometheus-community/kube-prometheus-stack \
7   -n observability \
8   -f kubernetes/prometheus-values.yaml \
9   --set "kubeControllerManager.endpoints=${NODE_IP}" \
10  --set "kubeScheduler.endpoints=${NODE_IP}" \
11  --set "kubeProxy.endpoints=${NODE_IP}" \
12  --wait --timeout 8m
```

Listing 4.2: Instalacja Prometheusa (`deploy-k8s.sh`)

Grafana Alloy zbiera wszystkie trzy typy sygnałów w ramach jednego agenta. Mikroserwis wysyła ślady rozproszone przez odbiornik OTLP:

```
1 otelcol.receiver.otlp "default" {
2   grpc { endpoint = "0.0.0.0:4317" }
3   http { endpoint = "0.0.0.0:4318" }
4   output { traces = [otelcol.processor.batch.default.input] }
```

```

5 }
6
7 otelcol.exporter.otlp "tempo" {
8   client {
9     endpoint = "tempo.observability.svc.cluster.local:4317"
10    tls { insecure = true }
11  }
12 }

```

Listing 4.3: Konfiguracja odbiornika OTLP w Alloy (alloy-values.yaml)

Korelacja sygnałów w Grafanie została skonfigurowana poprzez powiązanie źródeł danych – nawigację od metryk do traces, od traces do logów oraz mapę serwisów:

```

1 - name: Tempo
2   type: tempo
3   uid: tempo
4   url: http://tempo.observability.svc.cluster.local:3200
5   jsonData:
6     tracesToMetrics:
7       datasourceUid: prometheus
8     tracesToLogsV2:
9       datasourceUid: loki
10      filterByTraceID: true
11   serviceMap:
12     datasourceUid: prometheus
13   nodeGraph:
14     enabled: true

```

Listing 4.4: Konfiguracja datasource Tempo (grafana-values.yaml)

```
SUU-App-0 $ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
boutique	cartservice-7956c8c845-tlp57	1/1	Running	0	51m
boutique	cartservice-f586f5d69-blh4m	0/1	Pending	0	50m
boutique	checkoutservice-6b5dfd5cc4-sjh45	1/1	Running	0	50m
boutique	currencyservice-67f7bbd97b-78mff	1/1	Running	0	50m
boutique	emailservice-956b78dfd-fvrn9	1/1	Running	0	50m
boutique	frontend-79bbfcb49-pgkhz	1/1	Running	0	50m
boutique	paymentservice-b4bfb96db-rgxn7	1/1	Running	0	50m
boutique	productcatalogservice-5f94c4df4-7smbm	1/1	Running	0	50m
boutique	recommendationservice-898c4c86-pf86z	1/1	Running	0	50m
boutique	redis-cart-6899b65948-mwk2d	1/1	Running	0	51m
boutique	shippingservice-7589f54db-4tpf9	1/1	Running	0	50m
kube-system	coredns-8db54c48d-4cgbk	1/1	Running	0	55m
kube-system	helm-install-traefik-crd-79k65	0/1	Completed	0	55m
kube-system	helm-install-traefik-m7xkj	0/1	Completed	2	55m
kube-system	local-path-provisioner-5d9d9885bc-4xqlj	1/1	Running	0	55m
kube-system	metrics-server-786d997795-rnqfd	1/1	Running	0	55m
kube-system	svclb-frontend-external-c2f79784-lhb9g	0/1	Pending	0	51m
kube-system	svclb-traefik-a4a62997-j77qq	2/2	Running	0	54m
kube-system	traefik-9bcd9bd9-6ccx7	1/1	Running	0	54m
observability	alertmanager-kps-kube-prometheus-stack-alertmanager-0	2/2	Running	0	54m
observability	alloy-nskgf	2/2	Running	0	52m
observability	grafana-7b478976b5-xfsqp	1/1	Running	0	52m
observability	kps-kube-prometheus-stack-operator-d746b5dd7-pzkcq	1/1	Running	0	54m
observability	kps-kube-state-metrics-66655cbc7f-hwvrf	1/1	Running	0	54m
observability	kps-prometheus-node-exporter-fwq4b	1/1	Running	0	54m
observability	loki-0	2/2	Running	0	54m
observability	loki-canary-j6x4l	1/1	Running	0	54m
observability	mcp-grafana-56c98b5465-ztgrc	1/1	Running	0	50m
observability	prometheus-kps-kube-prometheus-stack-prometheus-0	2/2	Running	0	54m
observability	tempo-0	1/1	Running	0	53m

```
SUU-App-0 $
```

Rysunek 4.2: Pody systemu po wdrożeniu (`kubectl get pods -A`)

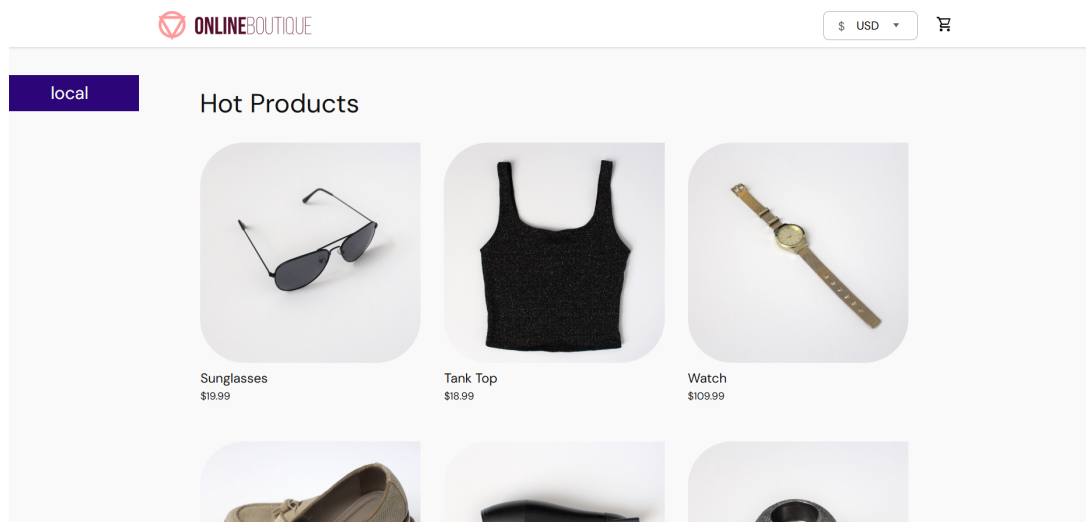
4.2.2 Aplikacja Online Boutique

Aplikacja została wdrożona z oficjalnych manifestów Kubernetes, uzupełnionych o usługę NodePort udostępniającą frontend na porcie 30080. W celu oszczędzenia zasobów usunięto `loadgenerator` i wyłączono `adservice`.

Integrację z warstwą obserwowalności zrealizowano przez wstrzyknięcie zmiennych OTEL do mikroserwisów:

```
1 for svc in frontend checkoutservice productcatalogservice \
2   shippingservice cartservice paymentservice emailservice \
3   recommendationservice currencyservice; do
4   kubectl set env deployment/$svc -n boutique \
5     OTEL_EXPORTER_OTLP_ENDPOINT=\
6     "http://alloy.observability.svc.cluster.local:4317" \
7     OTEL_EXPORTER_OTLP_INSECURE="true"
8 done
```

Listing 4.5: Konfiguracja OTEL w mikroserwisach



Rysunek 4.3: Interfejs aplikacji Online Boutique

4.3 Integracja z agentem AI

Integrację z agentem AI zrealizowano za pomocą Grafana MCP Server, wdrożonego skryptem `deploy-mcp.sh`. Skrypt tworzy konto serwisowe w Grafanie, generuje token API i zapisuje dane uwierzytelniające w Kubernetes Secret:

```
1 kubectl -n observability create secret generic \
2   mcp-grafana-credentials \
3   --from-literal=GRAFANA_URL="$GRAFANA_URL" \
4   --from-literal=GRAFANA_API_KEY="$GRAFANA_API_KEY" \
5   --dry-run=client -o yaml | kubectl apply -f -
```

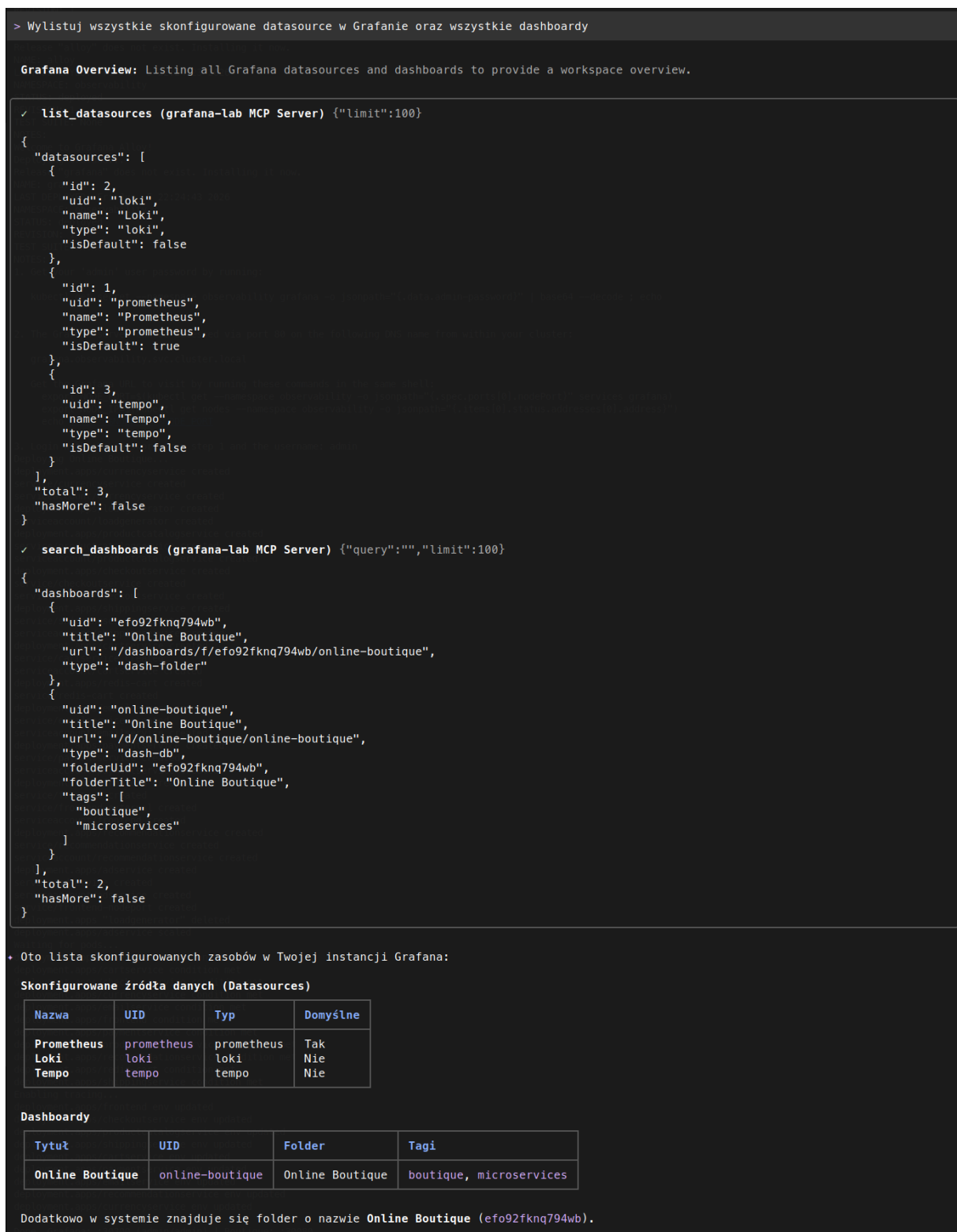
Listing 4.6: Tworzenie sekretu z tokenem API (`deploy-mcp.sh`)

MCP Server działa jako zasób typu Deployment i komunikuje się z Grafaną poprzez API REST:

```
1 spec:
2   containers:
3     - name: mcp-grafana
4       image: mcp/grafana:latest
5       args: ["-t", "sse", "-address", ":8000"]
6       env:
7         - name: GRAFANA_URL
8           valueFrom:
9             secretKeyRef:
10              name: mcp-grafana-credentials
11              key: GRAFANA_URL
12         - name: GRAFANA_API_KEY
13           valueFrom:
14             secretKeyRef:
15              name: mcp-grafana-credentials
16              key: GRAFANA_API_KEY
```

Listing 4.7: Fragment Deployment MCP Server (`mcp-grafana.yaml`)

Serwer jest dostępny na porcie **30090** w trybie SSE (Server-Sent Events). Agent AI (Gemini CLI) łączy się z endpointem `http://<IP>:30090/sse` i uzyskuje dostęp do danych telemetrycznych oraz dashboardów udostępnionych za pośrednictwem API serwera.



Rysunek 4.4: Agent AI (Gemini CLI) wywołujące serwer MCP w celu uzyskania informacji do odpowiedzi

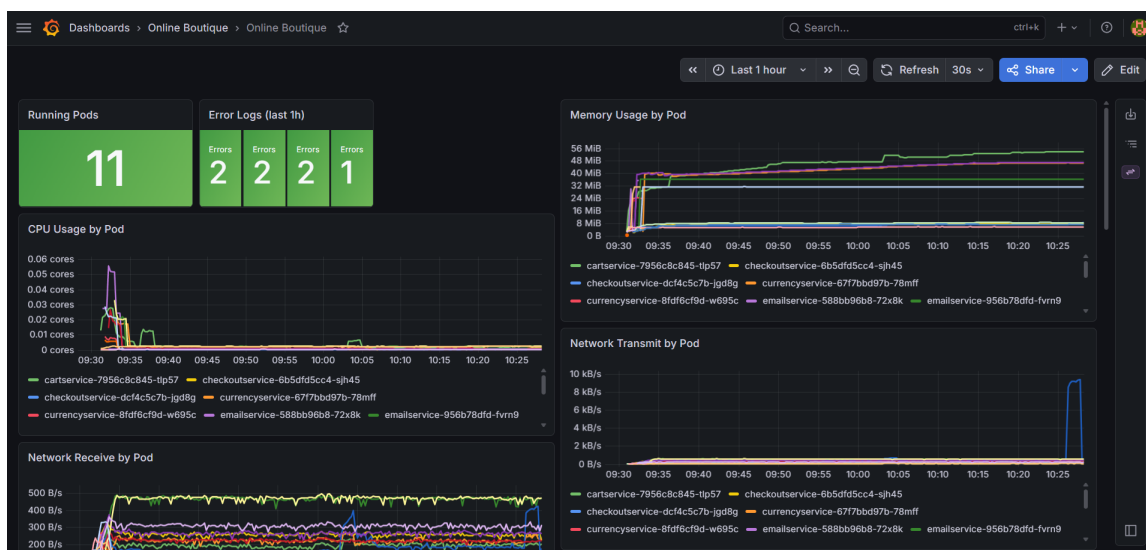
4.4 Dashboard Grafana

W systemie zdefiniowano dashboard **Online Boutique**, ładowany automatycznie z ConfigMap. Dashboard zawiera panele prezentujące kluczowe aspekty działania aplikacji:

Tabela 4.1: Panele dashboardu Online Boutique

Panel	Typ	Źródło
Running Pods	stat	Prometheus
Error Logs (last 1h)	stat	Loki
CPU Usage by Pod	timeseries	Prometheus
Memory Usage by Pod	timeseries	Prometheus
Network Receive/Transmit	timeseries	Prometheus
Pod Restarts	timeseries	Prometheus
Pod Status	table	Prometheus
Boutique Logs	logs	Loki
Error Logs Only	logs	Loki
Traces – Recent Requests	table	Tempo

Dzięki korelacji sygnałów możliwe jest przechodzenie z wykresu metryki do odpowiadających logów, a z logu do konkretnego śladu rozproszonego.



Rysunek 4.5: Dashboard Online Boutique w Grafanie

```

Boutique Logs

: DEBUG app=frontend instance=boutique/frontend-79bb... pod=frontend-79bbfcb49-pgkhz service_name=frontend {
  "http.req.id": "e94b75de-72c1-41b1-b6e9-8b834388a20a",
  "http.req.method": "GET",
  "http.req.path": "/_healthz",
  "http.resp.bytes": 2,
  "http.resp.status": 200,
  "http.resp.took_ms": 0,
  "message": "request complete",
  "session": "x-liveness-probe",
  "severity": "debug",
  "timestamp": "2026-06-04T08:28:42.767694839Z"
}

: DEBUG app=frontend instance=boutique/frontend-79bb... pod=frontend-79bbfcb49-pgkhz service_name=frontend {
  "http.req.id": "e94b75de-72c1-41b1-b6e9-8b834388a20a",
  "http.req.method": "GET",
  "http.req.path": "/_healthz",
  "http.resp.bytes": 2,
  "http.resp.status": 200,
  "http.resp.took_ms": 0,
  "message": "request complete",
  "session": "x-liveness-probe",
  "severity": "debug",
  "timestamp": "2026-06-04T08:28:42.767694839Z"
}

Error Logs Only

: WARN app=frontend instance=boutique/frontend-79bb... pod=frontend-79bbfcb49-pgkhz service_name=frontend {
  "error": "failed to get ads: rpc error: code = Unavailable desc = connection error: desc = \"transport: Error while dialing: dial tcp 10.43.203.23:9555: connec",
  "http.req.id": "df33e8a0-6c29-4368-bd72-2a7c1feef532",
  "http.req.method": "GET",
  "http.req.path": "/",
  "message": "failed to retrieve ads",
  "session": "290d4f0b-1473-4c9e-87dd-5ee2a6040310",
  "severity": "warn",
  "timestamp": "2026-06-04T08:28:42.767694839Z"
}

```

Rysunek 4.6: Widok logs – szczegóły konkretnych logów

Traces - Recent Requests						
	Trace ID	Start time	Service	Name		Duration
>	49b4e1df9d7c48df084f9cf02d3c52d	2026-06-04 10:09:41.912	unknown_service	/grpc.health.v1.Health/Check		<1 ms
>	53490e648201339d24d21c3b02a6c07	2026-06-04 10:09:39.612	unknown_service	/grpc.health.v1.Health/Check		<1 ms
>	454e90b08697b490762e796b752fab0	2026-06-04 10:09:38.747	paymentservice	grpc.grpc.health.v1.Health/Check		<1 ms
>	334d0ee38c0b86db37a30de44b7df98	2026-06-04 10:09:38.443	unknown_service:server	grpc.health.v1.Health/Check		<1 ms
>	6b746b0706dfef3cea4a372c3c0c9	2026-06-04 10:09:36.912	unknown_service	/grpc.health.v1.Health/Check		<1 ms
>	21e0068699230300ef1e80b37a82f005	2026-06-04 10:09:36.433	paymentservice	grpc.grpc.health.v1.Health/Check		<1 ms
>	668c461f121f66ab7e43d21e72c2469	2026-06-04 10:09:36.194	unknown_service:server	grpc.health.v1.Health/Check		<1 ms
>	72eb3a126cf5179ce21b858eb03eff87a	2026-06-04 10:09:33.165	unknown_service:server	opentelemetry.proto.collector.trace.v1		<1 ms

Rysunek 4.7: Widok traces – szczegóły śladu rozproszonego

4.5 Generowanie ruchu testowego

Do generowania danych telemetrycznych wykorzystano skrypt `load.sh`, który symuluje ruch użytkowników. Skrypt uruchamia konfigurowalną liczbę równoległych workerów, z których każdy cyklicznie wysyła żądania HTTP do losowego endpointu aplikacji.

```

1 ENDPOINTS=(
2   "/" "product/OLJCESPC7Z" "product/66VCHSJNUP"
3   "product/1YMWWN1N40" "product/L9ECAV7KIM" "cart"
4   "product/2ZFYJ3GM2N" "product/OPUK6V6EKW"
5   "product/LS4PSXUNUM"
6 )
7
8 worker() {
9   while true; do
10    ENDPOINT=${ENDPOINTS[$RANDOM % ${#ENDPOINTS[@]}]}
11    STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
12      --max-time 5 "$URL$ENDPOINT")
13    echo "GET $ENDPOINT [$STATUS]"
14    sleep "$DELAY"
15   done
16 }
17
18 for i in $(seq 1 $CONCURRENT); do

```



```

19     worker &
20 done
21 wait

```

Listing 4.8: Skrypt generowania ruchu (tests/load.sh)

Skrypt przyjmuje trzy parametry: adres aplikacji, liczbę workerów (domyślnie 3) oraz opóźnienie między żadaniami (domyślnie 0.5s). Przy domyślnych ustawieniach generuje ok. 6 żądań na sekundę.

```

> ./load.sh
Boutique load generator
URL: http://54.87.37.122:30080
Concurrent workers: 3
Delay: 0.5s

Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET /product/66VCHSJNUP [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/2ZYFJ3GM2N [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET /product/L9ECAV7KIM [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /cart [200]
Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/2ZYFJ3GM2N [200]
Successful: GET /cart [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /cart [200]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /product/L9ECAV7KIM [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/0LJCESPC7Z [200]
Successful: GET /cart [200]
Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /cart [200]
Successful: GET /cart [200]
Successful: GET /product/66VCHSJNUP [200]
Successful: GET /product/1YMMWN1N40 [200]
Error: GET /product/0PUK6V6EKW [500]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/0LJCESPC7Z [200]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET /cart [200]
Successful: GET /product/0LJCESPC7Z [200]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET /product/66VCHSJNUP [200]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET / [200]
Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/0LJCESPC7Z [200]
Successful: GET /product/66VCHSJNUP [200]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /product/0LJCESPC7Z [200]
Successful: GET /cart [200]
Successful: GET /product/2ZYFJ3GM2N [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/LS4PSXUNUM [200]
Successful: GET / [200]
Successful: GET /product/66VCHSJNUP [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/1YMMWN1N40 [200]
Successful: GET /product/66VCHSJNUP [200]
Successful: GET / [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET / [200]
Successful: GET /product/2ZYFJ3GM2N [200]
Error: GET /product/0PUK6V6EKW [500]
Successful: GET /product/L9ECAV7KIM [200]
Successful: GET /product/2ZYFJ3GM2N [200]
Successful: GET /product/1YMMWN1N40 [200]

```

Rysunek 4.8: Wyjście skryptu load.sh podczas generowania ruchu

4.6 Proces instalacji

Cały proces wdrożenia jest zautomatyzowany i realizowany jednym poleceniem `./init.sh`. Skrypt kolejno:

1. instaluje Terraform i Helm,
2. tworzy infrastrukturę AWS (`terraform apply`),
3. oczekuje na gotowość K3s i pobiera kubeconfig,
4. wdraża stos obserwowalności i aplikację (`deploy-k8s.sh`),
5. wdraża serwer MCP (`deploy-mcp.sh`),
6. wyświetla adresy URL do Grafany, Online Boutique i MCP Server.

4.7 Struktura repozytorium

```
1 SUU-App-0/  
2 |-- terraform/  
3 |   |-- main.tf  
4 |   |-- variables.tf  
5 |   |-- outputs.tf  
6 |   +-- user_data.sh  
7 |-- kubernetes/  
8 |   |-- prometheus-values.yaml  
9 |   |-- loki-values.yaml  
10 |   |-- tempo-values.yaml  
11 |   |-- alloy-values.yaml  
12 |   |-- grafana-values.yaml  
13 |   |-- online-boutique.yaml  
14 |   |-- boutique-dashboard.json  
15 |   +-- grafana-mcp-server/  
16 |       |-- mcp-grafana.yaml  
17 |       +-- mcp-grafana-service.yaml  
18 |-- tests/  
19 |   +-- load.sh  
20 |-- init.sh  
21 |-- deploy-k8s.sh  
22 |-- deploy-mcp.sh  
23 +-- README.md
```

Listing 4.9: Struktura repozytorium projektu

Rozdział 5

Podsumowanie

Celem projektu było zaprojektowanie i wdrożenie systemu obserwowalności dla aplikacji mikroservisowej, zintegrowanego z agentem AI za pośrednictwem protokołu Model Context Protocol (MCP).

W ramach realizacji projektu zbudowane zostało kompletne środowisko składające się z klastra Kubernetes (K3s) uruchomionego na instancji AWS EC2, aplikacji demonstracyjnej Online Boutique oraz stosu obserwowalności opartego na narzędziach Grafana Alloy, Prometheus, Loki i Tempo. Całość procesu wdrożenia została zautomatyzowana za pomocą Terraform, Helm oraz skryptów Bash, co umożliwia powtarzalne uruchomienie systemu jednym poleceniem.

Zrealizowany system zbiera wszystkie trzy filary obserwowalności – metryki, logi i ślady rozproszone – w ramach zunifikowanego agenta (Grafana Alloy) i prezentuje je w Grafanie z pełną korelacją sygnałów. Dzięki temu możliwe jest przechodzenie od anomalii widocznej na wykresie metryki, przez odpowiadające jej logi, aż do konkretnego śladu rozproszonego wskazującego przyczynę problemu.

Kluczowym elementem projektu jest integracja z agentem AI (Gemini CLI) poprzez Grafana MCP Server. Agent uzyskuje dostęp do danych telemetrycznych i dashboardów za pomocą języka naturalnego, co otwiera możliwość automatycznej diagnozy stanu systemu bez konieczności ręcznego przeglądania dashboardów.

Przeprowadzone testy z wykorzystaniem skryptu generującego ruch potwierdziły poprawność działania całego pipeline’u telemetrycznego – od emisji danych przez mikroservisy, przez kolekcję i przechowywanie, aż po wizualizację i dostęp przez agenta AI.

5.1 Możliwości rozwoju

Zrealizowany system stanowi bazę, którą można rozbudować w kilku kierunkach:

- **Automatyczna naprawa** – rozszerzenie agenta AI o zdolność nie tylko diagnozowania problemów, ale także podejmowania działań naprawczych, takich jak restart serwisu czy skalowanie replik.
- **Alerting** – konfiguracja reguł alertów w Grafanie z powiadomieniami na e-mail, uzupełniona o automatyczną analizę alertów przez agenta AI.

- **Profilowanie** – dodanie czwartego filaru obserwowalności (profilu) przy użyciu Grafana Pyroscope, co pozwoliłoby na analizę wydajności na poziomie kodu źródłowego.
- **Klaster wielowęzłowy** – przeniesienie systemu na klaster z wieloma węzłami w celu testowania scenariuszy awarii węzła i zachowania systemu obserwowalności w warunkach częściowej niedostępności.

Literatura

- [1] *Google Gemini*. URL: <https://gemini.google.com/app>.
- [2] *Google Online Botique*. URL: <https://github.com/GoogleCloudPlatform/microservices-demo>.
- [3] *Grafana*. URL: <https://grafana.com/>.
- [4] *Grafana Alloy*. URL: <https://grafana.com/docs/alloy/latest/>.
- [5] *Grafana MCP Server*. URL: <https://github.com/grafana/mcp-grafana>.
- [6] *Kubernetes*. URL: <https://kubernetes.io/>.
- [7] *Open Telemetry*. URL: <https://www.dynatrace.com/monitoring/technologies/opentelemetry/>.
- [8] *Prometheus*. URL: <https://prometheus.io/>.
- [9] Sam Suthar. “What is observability 2.0?” W: *Cloud Native Computing Foundation* (2025). URL: <https://www.cncf.io/blog/2025/01/27/what-is-observability-2-0/>.
- [10] Alan Zeichick. “What Is Cloud Native?” W: *Oracle* (2025). URL: <https://www.oracle.com/cloud/cloud-native/what-is-cloud-native/>.